

Part 2

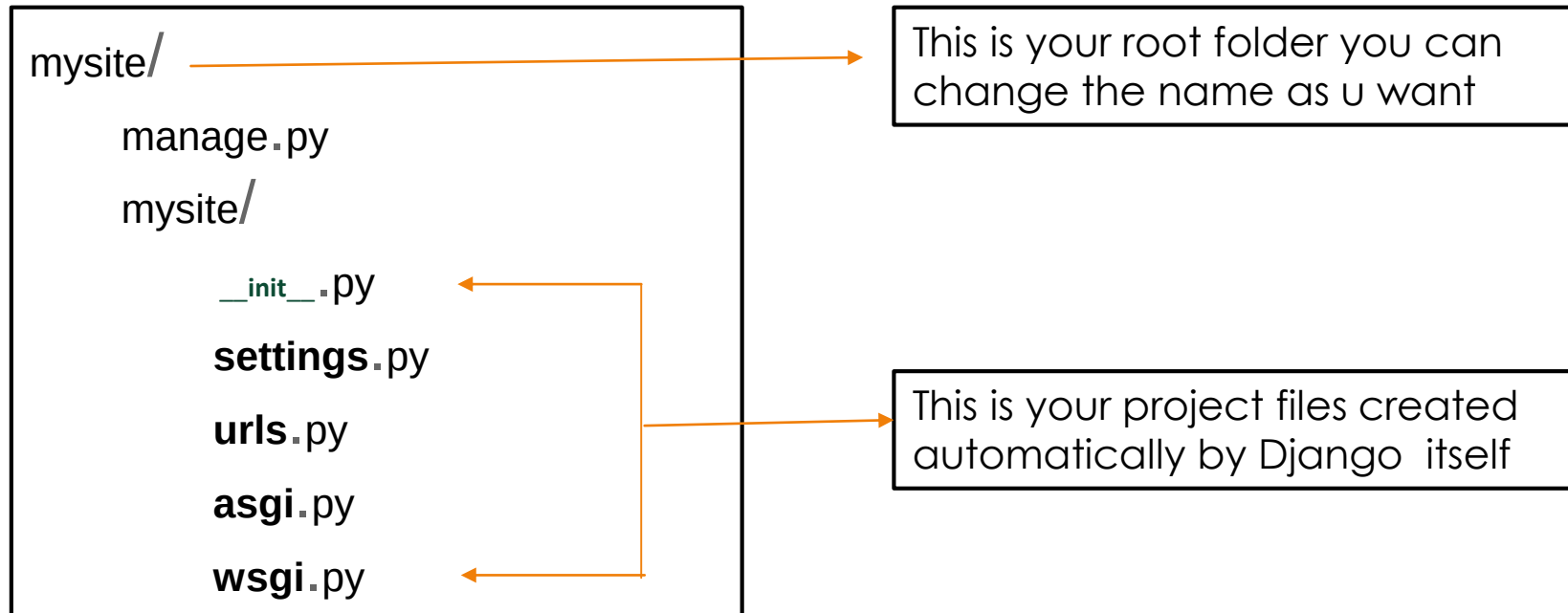
Django Project & Virtual Environment

VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

Understanding Django Project

Let's look at what **startproject** created:



Understanding Django Project

- 1) The outer **mysite/** root directory is a container for your project. Its name doesn't matter to Django; you can rename it to anything you like.
- 2) **manage.py** : A command-line utility that lets you interact with this Django project in various ways.
- 3) The inner **mysite/** directory is the actual Python package for your project. Its name is the Python package name you'll need to use to import anything inside it (e.g. `mysite.urls`).

Understanding Django Project

- 4) **mysite/__init__.py**: An empty file that tells Python that this directory should be considered a Python package.
- 5) **mysite/settings.py**: Settings/configuration for this Django project. Django settings will tell you all about how settings work.
- 6) **mysite/urls.py**: The URL declarations for this Django project; a “table of contents” of your Django-powered site.

Development Server Changing the Port

- ✓ By default, the `runserver` command starts the development server on the internal **IP** at port **8000**.
- ✓ If you want to change the server's port, pass it as a command-line argument. For instance, this command starts the server on port **8080** you can write whatever you want:

shell

// Run the project by 8080 or anything you want

```
>>> python manage.py runserver 8080
```

```
>>> python manage.py runserver 0.0.0.0:8000
```

Run the project by 8080
Or 80000

Screen shot of running the project

- ✓ The running projects looks like this by default:

```
Performing system checks...  
  
0 errors found  
January 08, 2020 - 15:50:53  
Django version 1.8, using settings 'mysite.settings'  
Starting development server at http://127.0.0.1:8000/  
Quit the server with CONTROL-C.
```

This is by default port of your project. You can also change if you want

Terminate the Server

- ✓ If you want to terminate the server hit the following command

```
// Terminate the server. This command help you to terminate the server  
>>> Hit : Ctrl + C  
  
// If your server is hang or not working well you can also used.  
>>> Hit : Ctrl + C
```

- ✓ If you clear the Django project running or terminating process. We can move the next step called virtual Environment.

What is Virtual Environment?

- ✓ Virtual Environment is a **Python** tool used for creating isolated environments.
- ✓ This separates each project from another so that the tools and actions active in one project environment do not have an effect in another environment except they are installed or activated in the other environment.
- ✓ You have create more than Environment in a single PC.

How to install Virtual Environment in PC?

- ✓ To install Virtual Environment, use the **pip** package manager that comes **pre-installed** with **Python**. Use the following command on the command prompt.

shell

```
>>> pip install virtualenv
```

```
>>> pip install virtualenvwrapper-win
```

Download virtual environment

- ✓ Go to “**C**” disk, clicks the “**users**”, click your “**user name**” in your pc search for “**Envs**”.
- ✓ Click the “Envs” for see your environment you already created.

How to make Virtual Environment? Method 1

- ✓ To create virtual environment write the following command.

shell

```
>>> mkvirtualenv "Your Environment Name"
```

```
// Example to create virtual env
```

```
>>> mkvirtualenv test
```

Make virtual environment

- ✓ The "mkvirtualenv" command help you to make the virtual environment in your PC.
- ✓ You can also name your virtual environment.

How to make Virtual Environment? Method 2

- ✓ To create virtual environment in second method write the following command.

shell

```
>>> virtualenv "Your Environment Name"
```

```
// Example to create virtual env
```

```
>>> virtualenv test2
```

Make virtual environment

- ✓ The “virtualenv” command help you to make the virtual environment in your PC.
- ✓ You can also name your virtual environment.

How to Activate Virtual Environment? Method 1

- ✓ To activate virtual environment write the following command.

```
shell
```

```
// write the "workon" word
```

```
>>> workon text
```

Activate Virtual Environment

- ✓ If your virtual environment is activated you can use its packages.

How to Activate Virtual Environment? Method 2

- ✓ Go to your virtual environment folder you created previously.
- ✓ Go to your **script** folder.
- ✓ Search for **activate** file if you found write the following command in **CMD**.

shell

// write the "workon" word

>>> scripts / activate

Activate Virtual Environment

- ✓ If your virtual environment is activated you can use its packages.

How to Deactivate Virtual Environment?

- ✓ If you want to deactivate virtual environment write the following command.
This command help for both methods 1 and 2.

```
shell
```

```
// write the "deactivate" word
```

```
>>> scripts / deactivate
```

This can help you to deactivate the server.

How to add “Requirements.txt” file in Virtual Env?

- ✓ Virtual Environments are, what I think, what really makes your requirements.txt powerful. Remember in the first section how we ran `pip freeze` and it listed all the packages on the entire system? Wouldn't it be better if it only listed that packages that we care about?
- ✓ This is one of the benefits of using virtual environments. When you are using a virtual environment, you only see the packages that you have installed in that environment. This helps prevent version conflicts between different projects. It also makes it easier to keep track of your packages.

How to add “Requirements.txt” file in Virtual Env?

- ✓ If you are using a virtual environment, and you want to create your requirements.txt, it is as simple as running this command:

shell

```
// Access your Virtual Environment
```

```
>>> pip freeze > requirements.txt
```

This can help you to install the package that can be used in your project.

How to install “requirements.txt” in another Env?

- ✓ If you want to install the “requirements.txt” on another environment write the following command in **CMD**.

shell

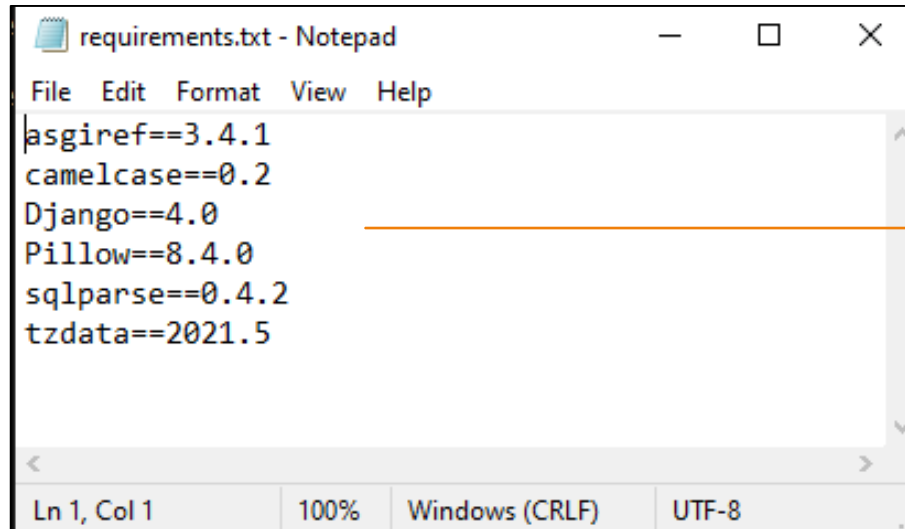
```
// install “requirements.txt” file in Env
```

```
>>> pip install -r requirements.txt
```

This can help you to install all the packages that you have installed previously in another Env.

Requirements.txt File

- ✓ Your “requirements.txt” file looks like:



A screenshot of a Notepad window titled "requirements.txt - Notepad". The window displays the following text:

```
asgiref==3.4.1  
camelcase==0.2  
Django==4.0  
Pillow==8.4.0  
sqlparse==0.4.2  
tzdata==2021.5
```

The status bar at the bottom indicates "Ln 1, Col 1", "100%", "Windows (CRLF)", and "UTF-8".

This is all your packages you have created with the help of **pip**.

Thanks for Reading